



Solar System Visualization

A cross-platform Flutter app: an interactive 2D pseudo-3D solar system, a historical astronomy timeline, a quiz, live NASA data, and an AI astronomy assistant.

[Technical Documentation — A to Z](#)

Live demo: solar-system-3d-viz.web.app
Prepared September 2026

Table of Contents

- A.** Overview
- B.** Origin & Design Philosophy
- C.** Features
- D.** Tech Stack
- E.** Project Structure
- F.** The Solar System Renderer
- G.** Camera System: Focus, Tracking & Inertia
- H.** Astronomical Timeline
- I.** Planet Info & Wikipedia Links
- J.** Quiz System
- K.** NASA Astronomy Picture of the Day
- L.** AI Astronomy Chat
- M.** Data Models
- N.** Persistence
- O.** Responsive Layout
- P.** Testing
- Q.** Getting Started (Local Development)
- R.** Deploying the App (Firebase Hosting)
- S.** Deploying the Chat Backend (Cloudflare Workers)
- T.** Configuration Reference
- U.** Security & Cost Considerations
- V.** Design Tradeoffs & Known Limitations
- W.** Troubleshooting Log
- X.** Roadmap Ideas
- Y.** Credits & License
- Z.** Appendix: Command Reference

A. Overview

Solar System Visualization is an interactive solar system explorer built in Flutter. A user can drag to rotate and tilt the camera, pinch or scroll to zoom, and tap any planet to fly the camera to it. Alongside the 3D-style view sits a historical astronomy timeline (selecting an event flies the camera to the planet it concerns), a Planet Info panel with facts and a Wikipedia link, a 10-question quiz with a persisted best score, a live NASA Astronomy-Picture-of-the-Day panel, and an AI chat assistant that can answer astronomy questions and is aware of whichever planet is currently selected.

The app runs from a single Dart codebase on **Android, iOS, Windows, macOS, Linux, and the Web**. The web build is deployed to Firebase Hosting; the AI chat feature is backed by a small Cloudflare Worker.

LIVE DEMO

<https://solar-system-3d-viz.web.app>

B. Origin & Design Philosophy

This project began as a React Three Fiber (WebGL) web application — a genuine 3D scene with a free-roaming camera, built on Three.js. It was rebuilt from scratch in Flutter rather than ported line-for-line, because a 1:1 port would have meant bridging Flutter to a WebGL/3D engine, which is a fragile, platform-inconsistent path (WebGL bridges behave differently — or not at all — across mobile, desktop, and web).

Instead, the solar system is rendered with a hand-rolled `CustomPainter` : a 2D canvas renderer using a tilted-ellipse projection to fake a 3D-ish view (the same technique classic astronomy diagrams use). This was a deliberate trade: it gives up a literal free-roaming 3D camera, in exchange for a renderer that behaves identically — no missing features, no platform-specific bugs — on every single platform Flutter targets, using only Flutter's own 2D graphics primitives.

Every later feature (the timeline's camera-fly-to effect, tap-to-focus, drag inertia, the quiz, live NASA data, and the AI chat) was added incrementally on top of that same foundation.

C. Features

Solar System View

The Sun, all 9 planets plus Pluto, Earth's Moon, Saturn's rings, a 240-body asteroid belt, and two animated space-mission trajectories (Voyager 1, New Horizons), all textured with real equirectangular maps and shaded relative to the Sun's position.

Camera Controls

Drag to rotate/tilt the view (with momentum after release), pinch or scroll to zoom, tap a planet to fly the camera to it.

Astronomical Timeline

13 historical events, Copernicus (1543) to the first black-hole image (2019). Selecting one pans/zooms the camera to the planet it concerns, or eases back to the full-system overview for general events.

Planet Info

Quick facts (distance, orbital period, temperature, diameter), a description, fun facts, and a "Learn More on Wikipedia" link per planet.

Quiz

10 questions, live scoring, an answer-review screen, and a best score that persists across sessions.

NASA APOD

Today's Astronomy Picture of the Day, fetched live from NASA's public API, with its full write-up and image.

AI Chat

An "Ask AI" panel — a conversational assistant for astronomy questions, aware of the currently selected planet.

Persistence

Last-viewed planet and best quiz score are remembered locally between sessions (`shared_preferences`).

Responsive Layout

A persistent timeline sidebar on wide screens; a draggable bottom sheet with five tabs on narrow/mobile screens.

Cross-Platform

One Dart codebase; Android, iOS, Windows, macOS, Linux, and Web builds from the same source.

D. Tech Stack

Layer	Choice	Why
App framework	Flutter / Dart	Single codebase across mobile, desktop, and web.
Solar system renderer	Hand-rolled <code>CustomPainter</code>	No external 3D/game engine; identical behavior on every platform. Core orbit/camera math is factored into pure top-level functions, unit-tested independently of the widget tree.
Live space data	NASA APOD public API (<code>http</code> package)	Free, no key required, demonstrates real external API integration.
Image CORS workaround	<code>images.weserv.nl</code> proxy (web only)	NASA's image host sends no CORS headers, which blocks <code>Image.network</code> specifically on Flutter Web.
AI chat backend	Cloudflare Worker + Workers AI	Runs an open LLM directly on Cloudflare's free tier — no API key, no billing account, anywhere.
Chat rate limiting	Cloudflare Workers KV	Per-IP hourly cap, persists across cold starts and edge locations.
Local persistence	<code>shared_preferences</code>	Best quiz score, last-viewed planet.
Outbound links	<code>url_launcher</code>	Wikipedia and NASA.gov links.
Web hosting	Firebase Hosting	Free static hosting for the compiled Flutter web build.

E. Project Structure

```
solar_system_flutter/
├─ lib/
│   ├─ main.dart                App entry point, MaterialApp + theme
│   ├─ models/
│   │   ├─ planet.dart          Planet + Moon data classes
│   │   ├─ historical_event.dart HistoricalEvent + type enum
│   │   ├─ quiz_question.dart    QuizQuestion data class
│   │   └─ apod_entry.dart       Parses NASA's APOD JSON response
│   ├─ data/
│   │   ├─ planets_data.dart     All 9 planets + Pluto, moons, textures
│   │   ├─ timeline_data.dart    13 historical events
│   │   └─ quiz_data.dart        10 quiz questions
│   ├─ services/
│   │   ├─ apod_service.dart     Fetches/caches NASA's APOD (per day)
│   │   └─ chat_service.dart     Calls the Cloudflare Worker chat endpoint
│   ├─ widgets/
│   │   ├─ solar_system_view.dart The CustomPainter renderer (core of the app)
│   │   ├─ controls_bar.dart      Pause/Play rotation button
│   │   ├─ planet_info_panel.dart Facts + Wikipedia link
│   │   ├─ timeline_panel.dart    Scrollable event list
│   │   ├─ quiz_panel.dart        Quiz UI + scoring
│   │   ├─ apod_panel.dart        APOD image + write-up
│   │   └─ chat_panel.dart        Chat UI (message bubbles, input)
│   ├─ screens/
│   │   └─ home_screen.dart       Layout, navigation, state orchestration
│   ├─ theme/
│   │   └─ app_theme.dart         Colors, Material theme
│   └─ utils/
│       ├─ external_link.dart     url_launcher wrapper
│       └─ app_prefs.dart         shared_preferences wrapper
├─ assets/textures/              Equirectangular planet/Sun/asteroid images
├─ test/
│   ├─ orbit_math_test.dart      Unit tests for orbit/camera-focus math
│   └─ widget_test.dart          Smoke test for the home screen
├─ web/                          index.html, manifest.json, icons, favicon
├─ android/ ios/ windows/ macos/ linux/ Platform scaffolding
├─ worker/
│   ├─ src/index.ts              Rate limit + Workers AI call
│   ├─ wrangler.toml             Worker config (KV + AI bindings)
│   └─ README.md                 Worker-specific setup instructions
├─ firebase.json, .firebaserc    Firebase Hosting config
├─ pubspec.yaml                  Flutter dependencies
└─ README.md                     Project README
```

F. The Solar System Renderer

`lib/widgets/solar_system_view.dart` is the heart of the app. It is a `StatefulWidget` wrapping a `CustomPainter`, driven by a `Ticker` that advances a simulation clock (`_simTime`) once per frame when auto-rotation is on.

Projection

Each planet's position is computed from a pure function:

```
double orbitAngleFor(Planet planet, double simTime) {  
  return (planet.name.hashCode % 360) * math.pi / 180 +  
    simTime * planet.orbitSpeed;  
}
```

The resulting angle and the planet's orbital radius are projected to screen space with a tilted-ellipse transform: the horizontal (x) component of the orbit maps directly to screen x, while the depth (z) component is compressed by a `tilt` factor before mapping to screen y. This is exactly how classic 2D solar-system diagrams simulate a 3D-ish oblique view without any real 3D math.

What gets drawn, in order

1. A radial-gradient space background and a twinkling starfield.
2. Orbit path rings for all planets (highlighted blue for the selected planet).
3. The asteroid belt (240 procedurally-seeded bodies between Mars and Jupiter).
4. Space-mission trajectories (quadratic Bézier curves with an animated marker).
5. The Sun (with a soft radial glow).
6. Planets, depth-sorted by screen Y so nearer-looking bodies draw on top — each with its own texture, a sun-relative shading overlay (a radial gradient positioned opposite the Sun direction), self-rotation, and a selection highlight ring/glow when selected.
7. Saturn's ring, drawn as a texture clipped to an annulus (outer ellipse minus inner ellipse via path difference).
8. Moons (currently just Earth's Moon), orbiting their parent planet's current position.

TESTABILITY

The orbit/camera math (`orbitAngleFor` , `computeZoomTarget` , `computePanTarget`) is deliberately factored out as top-level pure functions rather than private methods on the State class, specifically so it can be unit-tested without pumping a widget tree. See Section P.

G. Camera System: Focus, Tracking & Inertia

Manual controls

A `GestureDetector` wraps the whole view: `onScaleStart` / `onScaleUpdate` handle combined drag (rotate + tilt) and pinch (zoom) gestures — Flutter's scale gesture recognizer naturally unifies one-finger pan and two-finger pinch. `onScaleEnd` captures the release velocity and converts it into a decaying `_rotationVelocity`, applied and exponentially damped each tick — this is the drag-inertia effect.

Camera focus (the "fly to planet" effect)

Both tapping a planet directly and selecting a timeline event call the same underlying mechanism, `_applyFocus(planetName)`, which sets a target zoom (`computeZoomTarget` — closer planets get a tighter zoom, clamped to a sane range) and lets a target pan offset (`computePanTarget`) be recomputed every frame from the planet's live orbital position. Each tick, the actual `_zoom` and `_panOffset` ease toward these targets by a fixed fraction of the remaining distance — smooth, continuous "camera flies to the planet and then keeps following it" behavior, entirely driven by the same ticker that runs the simulation clock. Starting a manual drag cancels any active focus, handing control back to the user; the camera then eases back to the Sun-centered overview.

Pure math, tested in isolation

```
double computeZoomTarget(String? planetName, List<Planet> allPlanets);
Offset computePanTarget(String? planetName, List<Planet> allPlanets,
                        double simTime, double viewRotation);
```

These functions take no widget state — only plain values — so `test/orbit_math_test.dart` can assert their behavior directly (determinism, linear angle progression over time, closer-planet-means-more-zoom, and so on) without any Flutter test harness.

H. Astronomical Timeline

`lib/data/timeline_data.dart` holds 13 `HistoricalEvent` records, from the Heliocentric Theory (1543) to the first black-hole image (2019). Each event carries a `type` (theory / discovery / mission / observation, each with its own accent color), an optional `relatedPlanet`, and a `wikipediaUrl`.

Selecting an event:

1. Updates the selected planet (if the event has a `relatedPlanet`) and persists it.
2. Bumps a `focusToken` counter passed to `SolarSystemView`, which triggers the camera-focus system described in Section G — even re-selecting the same planet re-triggers the fly-to effect, because the token always changes.
3. Shows a `SnackBar` with a "Learn More" action that opens the event's Wikipedia page.
4. On mobile, closes the bottom sheet so the camera movement is immediately visible.

On wide screens, the timeline is also shown as a permanent sidebar (see Section O), letting a user watch the 3D-style view and browse history simultaneously.

I. Planet Info & Wikipedia Links

`lib/widgets/planet_info_panel.dart` shows the currently-selected planet's quick facts (distance from Sun, orbital period, surface temperature, diameter), a description, a bulleted list of fun facts, and a "Learn More on Wikipedia" button. Every planet in `planets_data.dart` carries its own `wikipediaUrl`, opened via `url_launcher` in an external browser tab — the app never embeds third-party content in a `WebView`.

J. Quiz System

`lib/widgets/quiz_panel.dart` runs a fixed 10-question multiple-choice quiz (`quiz_data.dart`). Answering a question locks it in and colors it green/red for correct/incorrect; a progress bar tracks position. On finishing, a review screen lists every question with the user's answer and the correct one, plus an overall score. The best score ever achieved is loaded on open and saved (only if it's a new high) via `AppPrefs` , so it persists across app restarts.

K. NASA Astronomy Picture of the Day

`lib/services/apod_service.dart` calls NASA's public APOD endpoint (`api.nasa.gov/planetary/apod` , using NASA's shared `DEMO_KEY`) and caches the result in memory for the current calendar day, so reopening the tab doesn't re-fetch. `lib/widgets/apod_panel.dart` renders the title, date, full write-up, optional copyright line, and the image itself, with loading and error states (including a retry button).

CORS PITFALL (FLUTTER WEB SPECIFIC)

NASA's image host does not send `Access-Control-Allow-Origin` headers. This silently breaks `Image.network` on Flutter *Web* — which needs to fetch the raw image bytes itself to decode them — while having no effect at all on native builds (CORS is purely a browser restriction; Android/iOS/desktop HTTP clients aren't subject to it). The fix, applied only when `kIsWeb` is true, routes the image URL through `images.weserv.nl` , a public CORS-friendly image proxy, leaving native platforms untouched and hitting the original URL directly.

L. AI Astronomy Chat

The "Ask AI" tab is a full chat interface (`lib/widgets/chat_panel.dart`) backed by `lib/services/chat_service.dart` , which calls a small Cloudflare Worker rather than any LLM API directly.

Why a backend proxy at all

NEVER CALL AN LLM API DIRECTLY FROM A SHIPPED CLIENT

Any API key embedded in a Flutter app — web or native — is extractable: on web it's visible in network requests and the compiled JS bundle; on native it can be pulled from the binary. A backend proxy is the only way to keep the key server-side while still letting the app use it.

The backend: from Firebase Functions to Cloudflare Workers to Workers AI

This went through two pivots before landing on the final design, each driven by a real constraint discovered along the way:

1. **First attempt — Firebase Cloud Functions calling the Claude API.** Abandoned because Cloud Functions require the Blaze (pay-as-you-go) billing plan even at zero usage — not viable without a card on file.
2. **Second attempt — a Cloudflare Worker calling the Claude API.** Cloudflare Workers has a genuine free tier with no card required, so the same proxy design (server-side key, IP rate limit) was rebuilt there using raw `fetch` to Anthropic's REST API. This worked mechanically, but every LLM provider's API is a metered, pay-per-token service — the Anthropic account behind the key had no funded credits, so every request failed with *"Your credit balance is too low."*
3. **Final design — the same Cloudflare Worker, using Workers AI instead of an external LLM API.** Workers AI runs open-source models directly on Cloudflare's own infrastructure, bound to the Worker via a simple `[ai] binding = "AI"` entry in `wrangler.toml` — no API key at all, no new account, and a genuine free daily allocation (10,000 Neurons/day, resets daily, no card).

Request flow

```
Flutter app (ChatService)
→ POST https://solar-system-chat.<subdomain>.workers.dev
  { message, history[], selectedPlanet? }
→ Cloudflare Worker
  1. Rate-limit check (Workers KV, 20 msgs/hour per IP)
  2. Validate/cap message length and history length
  3. env.AI.run("@cf/meta/llama-3.2-3b-instruct", { messages, max_tokens })
  4. Return { reply: "..." }
```

System prompt

The Worker prepends a system message describing the assistant's role (a friendly astronomy assistant embedded in this specific app), instructs it to keep replies conversational and concise (this is a compact chat panel, not a document), and to steer off-topic questions back to space. When the client sends a `selectedPlanet`, a one-line context note is appended so the assistant knows what the user is currently looking at.

Rate limiting

A Workers KV namespace (`RATE_LIMIT_KV`) stores a per-IP counter with an hourly TTL. This is the main defense against one visitor exhausting the shared free daily Neuron allocation for everyone else. The check fails open (allows the request) if KV itself has an issue, rather than blocking legitimate users on an infrastructure hiccup.

M. Data Models

Model	Key fields	Source
<code>Planet</code>	name, size, distanceFromSun, orbitalPeriod, diameter, averageTemp, rotationSpeed, orbitSpeed, texture, description, funFacts, wikipediaUrl, moons	<code>planets_data.dart</code> (static)
<code>Moon</code>	name, size, distance, texture	nested under a <code>Planet</code>
<code>HistoricalEvent</code>	date, title, description, type, relatedPlanet?, wikipediaUrl	<code>timeline_data.dart</code> (static)
<code>QuizQuestion</code>	question, options[], correctAnswer	<code>quiz_data.dart</code> (static)
<code>ApodEntry</code>	title, date, explanation, mediaType, imageUrl, nasaUrl, copyright?	parsed at runtime from NASA's JSON response

N. Persistence

`lib/utils/app_prefs.dart` wraps `shared_preferences` with two concerns:

- **Best quiz score** — saved only when a new score beats the stored one.
- **Last-viewed planet** — saved whenever the selected planet changes (via a direct tap or a timeline selection), and loaded once in `HomeScreen.initState` to restore it on the next launch.

On web, `shared_preferences` is backed by `localStorage`, scoped to the deployed origin.

O. Responsive Layout

`HomeScreen` branches on window width (a 900px breakpoint):

- **Wide (> 900px):** the solar system view fills the left side; a permanent 380px sidebar on the right shows the Astronomical Timeline plus the currently-selected event.
- **Narrow (mobile):** the solar system view fills the screen; five buttons at the bottom (Planet Info, Timeline, Quiz, Today, Ask AI) open a `DraggableScrollableSheet` with a horizontally-scrollable tab bar switching between an `IndexedStack` of the five corresponding panels. The sheet opens smaller on narrow screens (55% vs 82% of height) so the animated camera reaction to a timeline selection is still visible behind it.

P. Testing

```
flutter analyze
flutter test
```

`test/orbit_math_test.dart` — 11 unit tests against the pure orbit/camera-focus functions:

- `orbitAngleFor` is deterministic and advances linearly with simulation time at the planet's own orbit speed.
- `computeZoomTarget` returns the full-overview zoom for `null` /unknown planets, zooms in more for closer planets than farther ones, and always stays within its configured clamp range.
- `computePanTarget` returns the origin for `null` /unknown planets, matches an independently-computed polar-to-Cartesian projection for a known planet, and places farther planets farther from the origin.

`test/widget_test.dart` — a smoke test that pumps the full app and asserts the core UI (rotation button, all four/five tab-opening buttons) renders.

WHY PURE FUNCTIONS

Extracting the orbit/camera math out of the `State` class into top-level functions was a deliberate refactor specifically to make it unit-testable without pumping a widget tree — a good instinct whenever logic doesn't strictly need `this`.

Q. Getting Started (Local Development)

```
flutter pub get  
flutter run -d chrome      # or -d windows / -d macos / an attached device
```

Release build for web:

```
flutter build web --release
```

DEBUG MODE + MOUSE-TRACKER ASSERTION (FLUTTER WEB)

`flutter run -d chrome` (debug mode) can spam a benign-but-noisy `MissingPluginException` - adjacent mouse-tracker assertion error tied to a known Flutter framework issue (bottom sheets / ListViews animating under the mouse cursor). It's stripped out of release builds entirely. Prefer `flutter run -d chrome --release` for anything beyond quick UI iteration.

R. Deploying the App (Firebase Hosting)

The Firebase project is `solar-system-3d-viz`, on the free Spark plan (Hosting doesn't require Blaze).
`firebase.json` points hosting at `build/web` with an SPA rewrite (all routes serve `index.html`).

```
flutter build web --release  
firebase deploy --only hosting
```

Live at <https://solar-system-3d-viz.web.app>.

S. Deploying the Chat Backend (Cloudflare Workers)

One-time setup, from the `worker/` directory:

```
npm install
npx wrangler login                # browser OAuth – free account, no card
npx wrangler kv namespace create RATE_LIMIT_KV # paste the printed id into wrangler.toml
npx wrangler deploy              # first run also registers a workers.dev
subdomain
```

Redeploying after any code change:

```
npx wrangler deploy
```

Live at <https://solar-system-chat.belalsaqer.workers.dev>, referenced from `lib/services/chat_service.dart`'s `_endpoint` constant.

DIAGNOSING A DEPLOYED WORKER

`npx wrangler tail --format pretty` streams live logs from the deployed Worker while you send it a test request (e.g. via `curl`) — the fastest way to see the real error behind a generic client-side failure message.

T. Configuration Reference

firebase.json

```
{
  "hosting": {
    "public": "build/web",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [{ "source": "**", "destination": "/index.html" }]
  }
}
```

worker/wrangler.toml

```
name = "solar-system-chat"
main = "src/index.ts"
compatibility_date = "2026-01-01"

[[kv_namespaces]]
binding = "RATE_LIMIT_KV"
id = "e1b32b4a42b6488d9f938c2581a9e19d"

[ai]
binding = "AI"
```

Key pubspec.yaml dependencies

Package	Purpose
http	NASA APOD fetch, chat Worker calls
url_launcher	Outbound Wikipedia / NASA links
shared_preferences	Best score + last-viewed planet

U. Security & Cost Considerations

- **No LLM API key ships in the client.** The Flutter app only ever talks to the app's own Cloudflare Worker; the Worker itself needs no API key at all now that it uses Workers AI.
- **Secrets are set out-of-band.** Any secret (during the earlier Anthropic-backed design) is entered via `wrangler secret put` 's own hidden prompt — never typed into a chat transcript, a file, or a command-line argument, all of which would otherwise be logged somewhere.
- **Rate limiting** (Workers KV, 20 messages/hour/IP) protects the shared Workers AI free daily allocation from being exhausted by one visitor.
- **Firestore/Cloudflare KV client access** is fully locked down — only the Worker's own backend code touches it; no client SDK ever gets direct access.
- **Zero ongoing cost** for the entire stack as deployed: Firebase Hosting (Spark/free plan), Cloudflare Workers + KV + Workers AI (all within free-tier allocations for this app's expected traffic).

V. Design Tradeoffs & Known Limitations

Tradeoff	Reasoning
2D pseudo-3D renderer instead of true 3D (WebGL/OpenGL)	Reliability and identical behavior across every Flutter target, at the cost of a fully free-roaming 3D camera.
Small open-source chat model (Llama 3.2 3B) instead of a frontier LLM	Genuinely free (Cloudflare Workers AI) vs. a paid, metered API. Quality is adequate for factual astronomy Q&A in a compact chat panel; not on par with a frontier model for open-ended reasoning.
NASA APOD uses the shared public <code>DEMO_KEY</code>	No signup required, at the cost of a shared, fairly low rate limit (30 req/hour, 50/day per IP against NASA's API).
Chat rate limiting is a simple KV counter, not a hardened abuse system	Adequate for a portfolio-scale demo; a production system serving real traffic would want more (e.g. CAPTCHA, stricter per-session limits).

W. Troubleshooting Log

Real issues hit during development, and how they were diagnosed — kept here since they're the kind of thing that recurs.

1. Flutter Web debug-mode mouse-tracker assertion spam

Symptom: repeated `Another exception was thrown: Assertion failed` in the browser console, seemingly breaking interactivity. **Cause:** a known Flutter framework issue in debug mode, triggered by bottom sheets/ListsViews animating under the mouse cursor. **Fix:** run in `--release` mode, where asserts are stripped entirely.

2. NASA image not loading on Flutter Web only

Symptom: APOD text loaded fine; the image silently failed via the `errorBuilder` fallback, only on web. **Cause:** NASA's image host sends no CORS headers, and Flutter Web's `Image.network` needs to fetch raw bytes itself (unlike a plain HTML `<img>` tag, which can display cross-origin images without CORS). **Fix:** route the image URL through `images.weserv.nl` on web only (`kIsWeb` check), leaving native platforms untouched.

3. Firebase Cloud Functions require Blaze even at zero usage

Symptom: couldn't deploy a Cloud Function without a billing account attached. **Fix:** moved the backend to Cloudflare Workers, which has a genuine no-card free tier.

4. Empty Cloudflare secret value

Symptom: the deployed Worker returned `401 x-api-key header is required` even after `wrangler secret put` reported success. **Diagnosis:** a temporary debug log (`typeof env.SECRET` , its length, and an 8-character prefix — never the full value) revealed the secret was bound but had zero length. **Cause:** the interactive hidden-input prompt didn't actually capture the pasted value on the first attempt (likely a terminal paste-timing issue). **Fix:** re-ran `wrangler secret put` , this time confirming the prompt showed masked characters before pressing Enter.

5. Anthropic account had no funded credits

Symptom: after fixing the empty-secret issue, requests failed with `400 Your credit balance is too low` . **Cause:** no LLM provider offers free API access — this is a real, unavoidable cost for any external LLM API. **Fix:** switched to Cloudflare Workers AI (Section L), which runs a model on Cloudflare's own infrastructure at no cost.

6. Orphaned background processes locking build output

Symptom: `flutter clean` failed with "Device or resource busy" on `build/web`. **Cause:** several earlier `flutter run -d chrome` sessions had not fully terminated their child `dart.exe` / `dartvm.exe` processes. **Fix:** identified and terminated the specific orphaned dev-tooling processes (never any user-facing browser window) via their command lines, then retried.

X. Roadmap Ideas

Not built, but considered reasonable next steps if the project continues:

- A GitHub Actions workflow running `flutter analyze` and `flutter test` on every push.
- An MIT `LICENSE` file for the public repository.
- Streaming chat responses instead of a single request/response round-trip.
- A larger or fine-tuned Workers AI model, if quality ever needs to improve beyond what Llama 3.2 3B offers.

Y. Credits & License

- Planet, Sun, and asteroid textures are sourced from the original project's asset set.
- Astronomy Picture of the Day content and imagery is provided by [NASA's APOD API](#); image credit (where provided) is shown alongside each entry.
- The AI chat assistant runs on [Cloudflare Workers AI](#) (Meta's Llama 3.2 3B Instruct model).

Z. Appendix: Command Reference

Flutter app

```
flutter pub get           # install dependencies
flutter analyze           # static analysis
flutter test              # run unit + widget tests
flutter run -d chrome --release # run in Chrome, release mode
flutter build web --release # production web build
firebase deploy --only hosting # deploy build/web to Firebase Hosting
```

Cloudflare Worker (from worker/)

```
npm install
npx wrangler login
npx wrangler kv namespace create RATE_LIMIT_KV
npx wrangler deploy
npx wrangler tail --format pretty # live logs from the deployed Worker
npx wrangler secret list          # list secrets bound to the Worker
npx wrangler secret delete NAME   # remove a secret
```

Solar System Visualization — Technical Documentation. Generated from the project's actual codebase and deployment history.